

Prototipo Zoom

Sistema de Videoconferencia por Sockets

PC4 - SW403U - Programación Orientada a Objetos

Universidad Nacional de Ingeniería

Facultad de Ingeniería Industrial y Sistemas

Integrantes:

Yampufe Pau Héctor Alonso - 20240310I

Romaní Cuyo Gabriel Alberto - 20242045K

Tasayco Portello Luis Antonio - 20240307H

Docente: CISNEROS NAPRAVNIK YAN EDUARDO

26 de junio de 2026

Índice

1. Introducción	4
2. Objetivos	4
3. Tecnologías Utilizadas	4
4. Arquitectura del Sistema	5
4.1. Requisitos Funcionales	7
4.2. Requisitos No Funcionales	7
5. Base de Datos	7
6. Diagrama Entidad-Relación	8
7. Documentación de Casos de Uso	9
7.1. Diagrama de Casos de Uso	9
7.2. Inventario de Casos de Uso	10
7.3. Especificaciones Detalladas	10
8. Diccionario de Datos	12
9. Diagrama de Secuencia	14
10. Diagrama de Actividades	15
11. Diagrama de Clases	16
12. Descripción del Código	17
12.1. Estructura del Proyecto	17
12.2. Arquitectura de Módulos	18
12.3. Clases Principales	18
12.4. Flujo de Ejecución	19
12.5. Funcionamiento de las Clases y Componentes	20
12.5.1. Lado del Servidor	20
12.5.2. Lado del Cliente	21
12.5.3. Gestores de Red y Hardware	22

12.6. Aplicación de los Pilares de la POO	22
12.6.1. Encapsulamiento	22
12.6.2. Herencia	23
12.6.3. Polimorfismo	23
12.6.4. Abstracción	23
12.7. Patrones de Diseño Implementados	24
12.7.1. Singleton (Instancia Única)	24
12.7.2. Observer (Observador)	24
12.7.3. Strategy (Estrategia)	25
12.7.4. Facade (Fachada)	25
12.7.5. Command (Comando)	25
12.7.6. Factory Method (Método Fábrica)	26
12.7.7. Template Method (Método Plantilla)	26
12.7.8. Resumen de Patrones	26
12.8. Manejo de Errores	26
12.9. Pruebas Realizadas	28
13.Concurrencia	28
13.1. Modelo de Concurrencia del Servidor	28
13.2. Concurrencia en el Cliente	29
14.Seguridad	29
14.1. Almacenamiento de Contraseñas	30
14.2. Validación de Autenticidad	30
15.Limitaciones Actuales	31
16.Protocolos y Normativas Utilizados	32
16.1. Tabla de Mensajes del Protocolo	32
17.Manual de Instalación y Ejecución	32
17.1. Requisitos	32
17.1.1. 1. Librerías de Python	32
17.1.2. 2. Dependencias del Sistema	34
17.2. Ejecución	34

17.3. Datos de Prueba	35
18. Conclusiones	35
19. Anexos	35
19.1. Repositorio del Proyecto	35

1. Introducción

El presente documento describe el diseño e implementación de un **prototipo académico de videoconferencia** inspirado en Zoom, desarrollado en Python como parte del curso de Programación Orientada a Objetos (PC4).

El sistema emplea una arquitectura **cliente-servidor** basada en **sockets TCP** con un protocolo de mensajes en formato **JSON**. Cuenta con autenticación de usuarios, creación y gestión de salas virtuales, chat en tiempo real, transferencia de archivos y transmisión de video desde la cámara web.

2. Objetivos

- Aplicar los principios de la POO: herencia, encapsulamiento, polimorfismo.
- Implementar una arquitectura cliente-servidor mediante sockets TCP.
- Desarrollar un protocolo de comunicación propio basado en JSON.
- Integrar múltiples funcionalidades: login, salas, chat, archivos y video.
- Utilizar SQLite para la persistencia de información.
- Emplear concurrencia con **threading** para atender múltiples clientes.

3. Tecnologías Utilizadas

- **Lenguaje:** Python 3.10+
- **Interfaz gráfica:** CustomTkinter
- **Comunicación:** Sockets TCP (RFC 793)
- **Protocolo de mensajes:** JSON sobre TCP (RFC 8259), delimitado por \n
- **Base de datos:** SQLite 3
- **Concurrencia:** threading (modelo de hilos POSIX)
- **Video:** OpenCV 4.x (backend MJPEG) o ffmpeg (V4L2 en Linux)

- **Audio:** PortAudio v19 mediante `sounddevice` (PCM 16 bit, 16 kHz mono)
- **Archivos:** transferencia binaria codificada en Base64 (RFC 4648)
- **Seguridad:** hash SHA-256 para contraseñas (FIPS 180-4)
- **Imágenes:** Pillow (PIL Fork)

4. Arquitectura del Sistema

El sistema sigue un modelo cliente-servidor. El servidor centraliza las conexiones y actúa como intermediario. Cada cliente se conecta mediante TCP y el servidor asigna un hilo independiente por conexión.

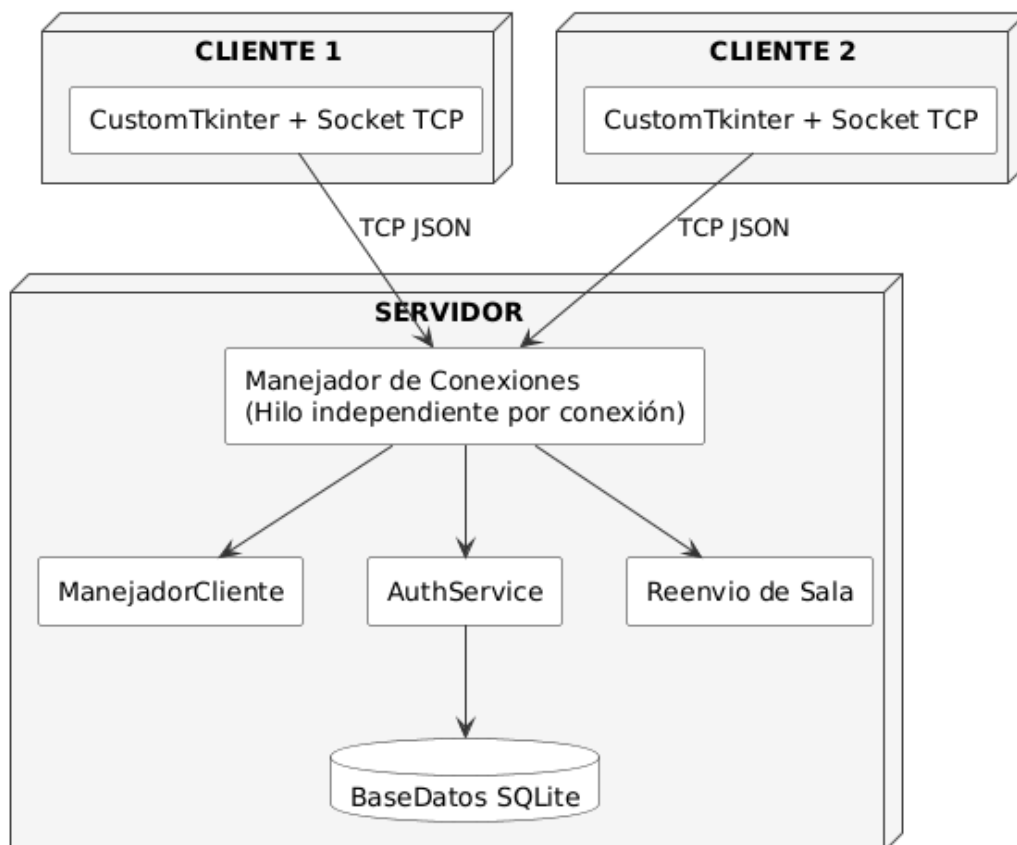


Figura 1: Diagrama de arquitectura del sistema

4.1. Requisitos Funcionales

Código	Descripción
RF01	El usuario puede iniciar sesión con correo y contraseña.
RF02	El usuario autenticado puede crear una sala de reunión.
RF03	El usuario autenticado puede unirse a una sala mediante un código.
RF04	El host puede admitir o rechazar solicitudes de ingreso a la sala.
RF05	Los participantes pueden enviar y recibir mensajes de chat en tiempo real.
RF06	Los participantes pueden compartir archivos dentro de la sala.
RF07	Los participantes pueden transmitir video desde su cámara web.
RF08	Los participantes pueden transmitir audio desde su micrófono.

Cuadro 1: Requisitos funcionales del sistema

4.2. Requisitos No Funcionales

Código	Descripción
RNF01	La comunicación se realiza mediante sockets TCP con mensajes JSON.
RNF02	Las contraseñas se almacenan con hash SHA-256, nunca en texto plano.
RNF03	El servidor soporta múltiples clientes simultáneos mediante hilos.
RNF04	La interfaz gráfica utiliza CustomTkinter con actualizaciones asíncronas.
RNF05	El sistema es multiplataforma (Windows y Linux).
RNF06	La base de datos SQLite es autocontenida y no requiere configuración externa.

Cuadro 2: Requisitos no funcionales del sistema

5. Base de Datos

La base de datos SQLite contiene las siguientes tablas:

Tabla	Columnas
Usuarios	IdUsuario, Nombres, Correo, PasswordHash, Rol, Activo
Salas	IdSala, CodigoSala, Nombre, IdHost, Estado, FechaCreacion
ParticipantesSala	IdParticipante, IdSala, IdUsuario, Estado, FechaIngreso
SolicitudesSala	IdSolicitud, IdSala, IdUsuario, Estado, FechaSolicitud
Mensajes	IdMensaje, IdSala, IdUsuario, Contenido, FechaEnvio
ArchivosCompartidos	IdArchivo, IdSala, IdUsuario, NombreArchivo, RutaArchivo, Tamano, FechaSubida

Cuadro 3: Tablas de la base de datos

6. Diagrama Entidad-Relación

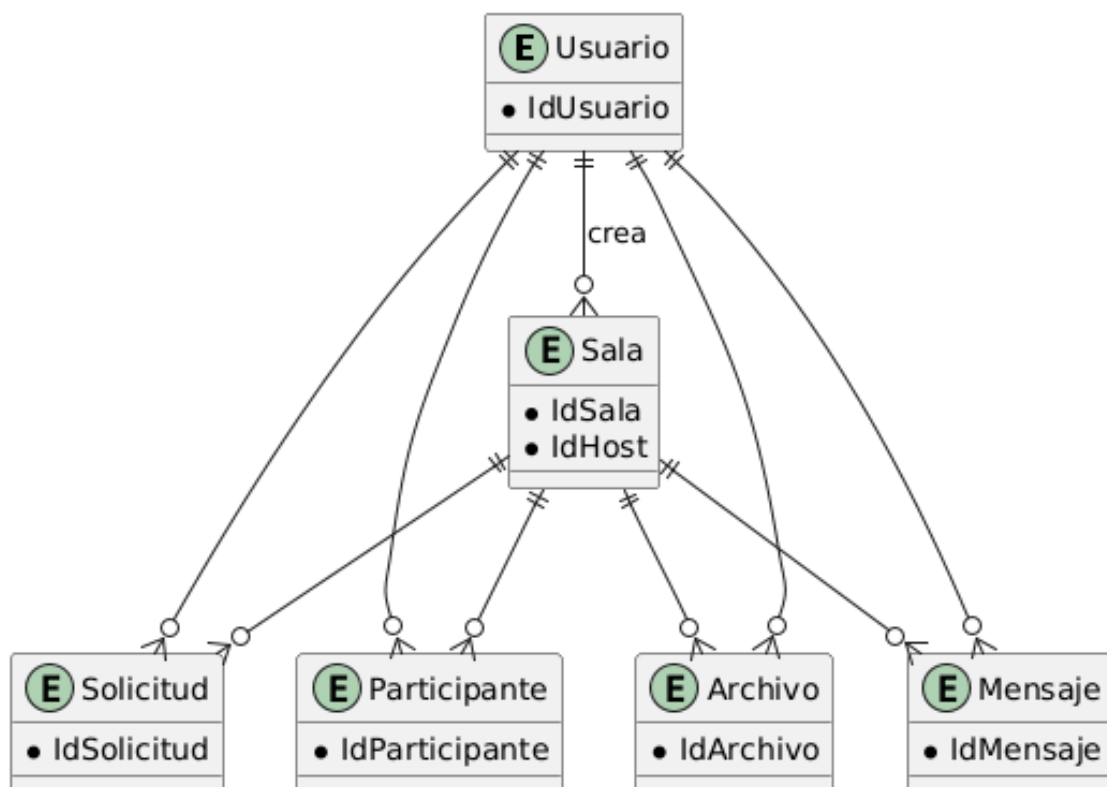


Figura 2: Diagrama Entidad-Relación de la base de datos

7. Documentación de Casos de Uso

7.1. Diagrama de Casos de Uso

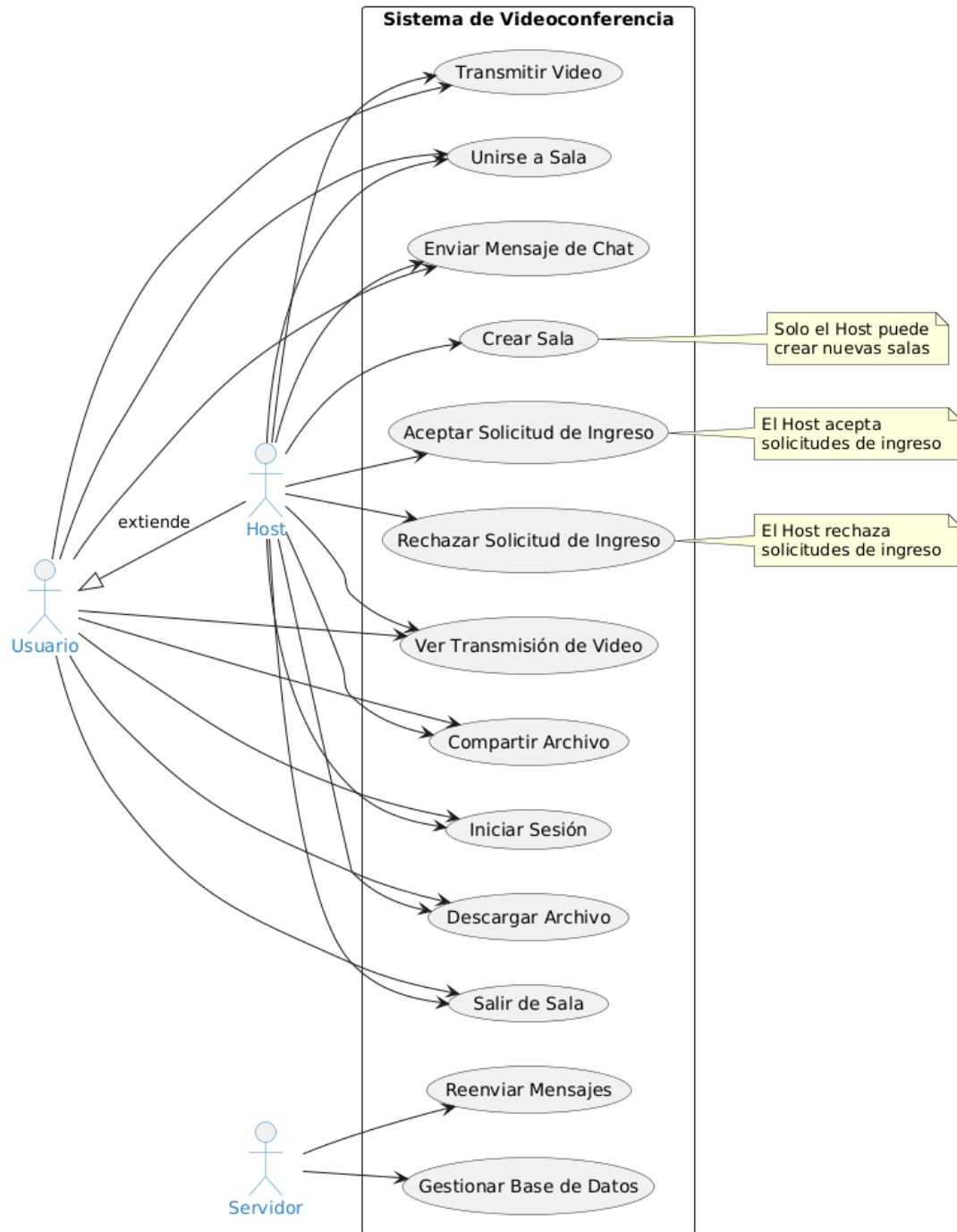


Figura 3: Diagrama de casos de uso del sistema

7.2. Inventario de Casos de Uso

A continuación se presenta la lista general y descripción breve de los casos de uso que componen el prototipo:

Código	Caso de Uso	Descripción Breve
CU-01	Iniciar Sesión	Permite al usuario autenticarse en el sistema mediante su correo y contraseña.
CU-02	Crear Sala	Permite a un anfitrión (Host) abrir una nueva sala virtual para una reunión.
CU-03	Unirse a Sala	Permite a un participante ingresar el código de una sala activa y solicitar acceso.
CU-04	Enviar Archivo	Permite compartir un documento dividiéndolo en fragmentos para su descarga.
CU-05	Transmitir Video	Permite capturar y enviar frames comprimidos de la cámara local a los demás participantes.
CU-06	Transmitir Audio	Permite capturar y reproducir flujos de voz en tiempo real desde el micrófono.
CU-07	Expulsar Participante	Permite al anfitrión expulsar de la sala activa a un participante admitido.
CU-08	Finalizar Sala	Permite al anfitrión cerrar formalmente la reunión para todos los integrantes.

Cuadro 4: Inventario de casos de uso del prototipo

7.3. Especificaciones Detalladas

Se describen a continuación los flujos detallados de los casos de uso principales del sistema.

CU-01: Iniciar Sesión

Campo	Descripción
Actor	Usuario registrado
Precondición	El servidor está activo y el usuario tiene cuenta en el sistema
Flujo principal	1. El usuario ingresa correo y contraseña. 2. El cliente envía <code>LOGIN_REQUEST</code> al servidor. 3. El servidor valida las credenciales con hash SHA-256. 4. El servidor responde con <code>LOGIN_RESPONSE</code> (<code>status: success</code>). 5. El cliente muestra la pantalla principal.
Flujo alternativo	Si las credenciales son incorrectas, el servidor responde con <code>status: error</code> y se muestra un mensaje al usuario.
Postcondición	El usuario queda autenticado y puede crear o unirse a salas.

Cuadro 5: CU-01: Iniciar Sesión

CU-02: Crear Sala

Campo	Descripción
Actor	Usuario autenticado (rol Host)
Precondición	El usuario ha iniciado sesión exitosamente
Flujo principal	1. El usuario presiona “Crear Sala”. 2. El cliente genera un código alfanumérico de 6 caracteres. 3. El cliente envía <code>CREATE_ROOM</code> con código y nombre. 4. El servidor registra la sala en la BD y asigna al usuario como host. 5. El cliente accede directamente a la sala como administrador.
Flujo alternativo	Si el código ya existe, el servidor retorna error y el cliente intenta con otro código.
Postcondición	La sala queda activa en la base de datos y el host puede gestionar participantes.

Cuadro 6: CU-02: Crear Sala

CU-03: Unirse a Sala

Campo	Descripción
Actor	Usuario autenticado
Precondición	Existe una sala activa con el código ingresado
Flujo principal	1. El usuario ingresa el código de sala y presiona “Unirse”. 2. El cliente envía <code>JOIN_ROOM_REQUEST</code>. 3. El servidor notifica al host con <code>WAITING_ROOM_UPDATE</code>. 4. El host admite al usuario (<code>ADMIT_USER</code>). 5. El servidor notifica al invitado y lo agrega a la sala.
Flujo alternativo	Si el host rechaza (<code>REJECT_USER</code>), el cliente recibe notificación de rechazo y regresa al menú.
Postcondición	El usuario es participante activo de la sala.

Cuadro 7: CU-03: Unirse a Sala

CU-04: Enviar Archivo

Campo	Descripción
Actor	Participante de la sala
Precondición	El usuario está dentro de una sala activa
Flujo principal	1. El usuario presiona “Archivo” y selecciona un fichero. 2. El cliente codifica el archivo en Base64 y lo fragmenta en chunks de 1500 bytes. 3. Envía <code>FILE_START</code>, los <code>FILE_CHUNK</code> y finalmente <code>FILE_END</code>. 4. Los receptores son consultados si desean descargar el archivo. 5. El receptor acepta y el archivo se reconstruye y guarda en descargas/.
Flujo alternativo	Si el receptor rechaza la descarga, el archivo no se guarda en ese cliente.
Postcondición	El archivo queda disponible en el equipo de los receptores que aceptaron.

Cuadro 8: CU-04: Enviar Archivo

8. Diccionario de Datos

Se describe cada atributo relevante de las entidades principales del sistema.

Tabla: Usuarios

Campo	Tipo	Descripción
IdUsuario	INTEGER	Clave primaria autoincrementable. Identificador único del usuario.
Nombres	TEXT	Nombre completo del usuario. No puede ser nulo.
Correo	TEXT	Dirección de correo electrónico. Debe ser única en el sistema.
PasswordHash	TEXT	Hash SHA-256 de la contraseña en hexadecimal. Nunca se almacena texto plano.
Rol	TEXT	Rol del usuario: <code>Host</code> o <code>Usuario</code> . Valor por defecto: <code>Usuario</code> .
Activo	INTEGER	Indicador booleano: 1 = activo, 0 = inactivo (baja lógica).

Cuadro 9: Diccionario de datos – Usuarios

Tabla: Salas

Campo	Tipo	Descripción
IdSala	INTEGER	Clave primaria autoincrementable.
CodigoSala	TEXT	Código alfanumérico de 6 caracteres generado en el cliente. Debe ser único.
Nombre	TEXT	Nombre descriptivo de la sala.
IdHost	INTEGER	Clave foránea a Usuarios . Usuario que creó y administra la sala.
Estado	TEXT	Estado de la sala: Activa o Cerrada . Valor por defecto: Activa .
FechaCreacion	TEXT	Fecha y hora de creación en formato ISO 8601 (UTC).

Cuadro 10: Diccionario de datos – Salas

Tabla: SolicitudesSala

Campo	Tipo	Descripción
IdSolicitud	INTEGER	Clave primaria autoincrementable.
IdSala	INTEGER	Clave foránea a Salas . Sala a la que se solicita ingreso.
IdUsuario	INTEGER	Clave foránea a Usuarios . Usuario que solicita ingresar.
Estado	TEXT	Estado de la solicitud: Pendiente , Admitido o Rechazado .
FechaSolicitud	TEXT	Fecha y hora de la solicitud en formato ISO 8601.

Cuadro 11: Diccionario de datos – SolicitudesSala

9. Diagrama de Secuencia

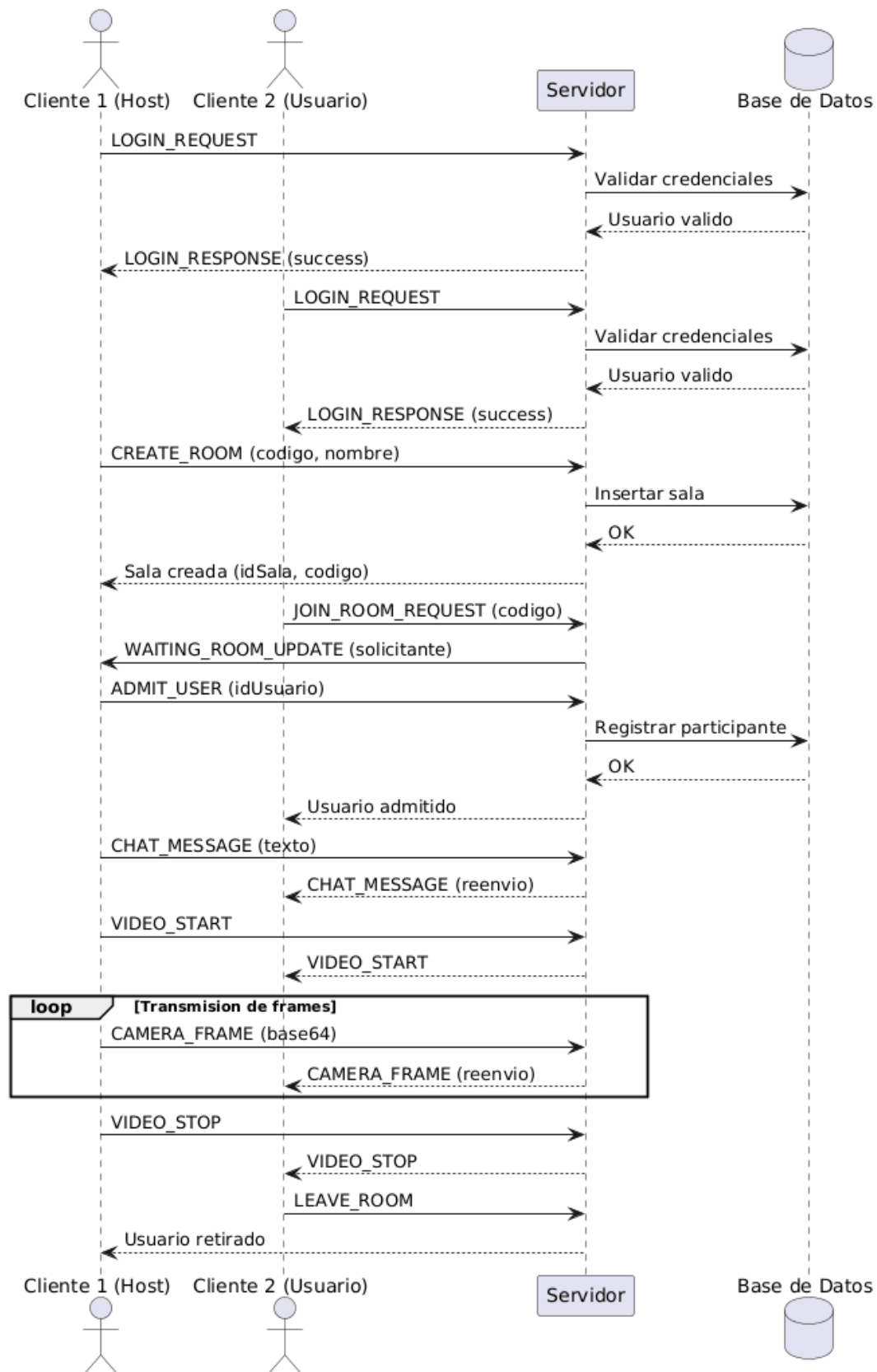


Figura 4: Diagrama de secuencia del flujo de la aplicación

10. Diagrama de Actividades

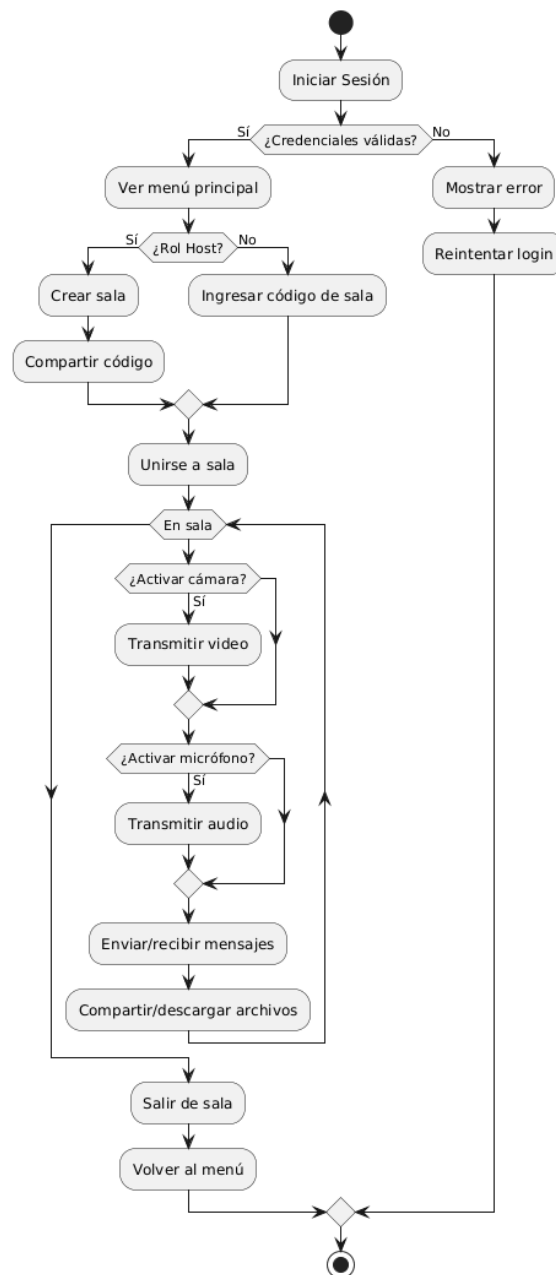


Figura 5: Diagrama de actividades del sistema

11. Diagrama de Clases

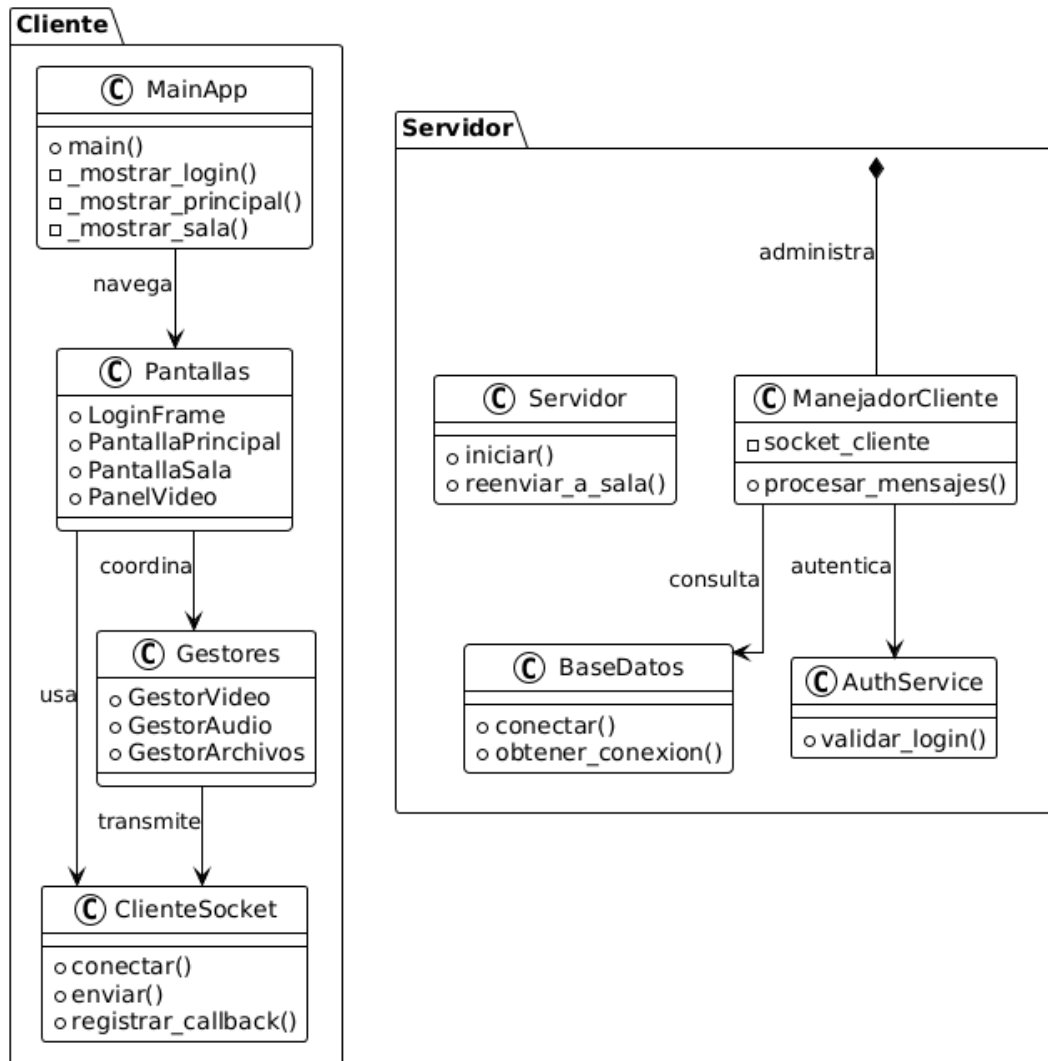


Figura 6: Diagrama de clases del sistema

12. Descripción del Código

Esta sección explica cómo se organiza el proyecto, sus clases principales, el flujo de ejecución, la aplicación de la POO, los patrones de diseño, el manejo de errores y las pruebas realizadas.

12.1. Estructura del Proyecto

El proyecto sigue una arquitectura cliente-servidor con separación clara de responsabilidades en módulos independientes. La estructura de directorios es la siguiente:

```
ProyectoZoom/
|
+-- Servidor/
|   +-- servidor.py           # Punto de entrada del servidor TCP
|   +-- manejador_cliente.py  # Manejador por conexion de cliente
|   +-- protocolo.py         # Constantes del protocolo de mensajes
|   +-- base_datos.py        # Singleton de conexion SQLite
|   +-- auth_service.py      # Validacion de credenciales
|   +-- __init__.py
|
+-- Cliente/
|   +-- main.py               # Punto de entrada, ventana principal
|   CustomTkinter
|   +-- cliente_socket.py     # Socket TCP con hilo listener y callbacks
|   +-- config.json           # Configuracion de servidor
|   |
|   +-- modelos/
|   |   +-- usuario.py        # Modelo de datos del usuario
|   |   +-- mensaje.py       # Modelo de datos del mensaje
|   |
|   +-- pantallas/
|   |   +-- login.py          # Pantalla de inicio de sesion
|   |   +-- pantalla_principal.py # Menu principal (crear/unirse a sala)
|   |
|   |   +-- pantalla_sala.py  # Pantalla de sala de reunion
|   |   +-- pantalla_video.py # Panel de video individual
|   |
|   +-- gestores/
```

```
|      +-- gestor_video.py    # Captura y transmision de video
|      +-- gestor_audio.py    # Captura y reproduccion de audio
|      +-- gestor_archivos.py # Transferencia de archivos
|
+-- BaseDatos/
|  +-- crear_tablas.sql        # Esquema de la base de datos
|  +-- datos_prueba.sql       # Datos de prueba iniciales
|
+-- Documentacion/           # Diagramas e informe LaTeX
+-- server.db                 # Base de datos SQLite (generada en
    ejecucion)
+-- requirements.txt          # Dependencias del proyecto
```

Esta separación por módulos permite reducir el acoplamiento, facilitando el mantenimiento y futuras ampliaciones del sistema.

12.2. Arquitectura de Módulos

El sistema se organiza en tres capas principales:

- **Servidor:** Gestiona conexiones entrantes, autenticación, registro de salas y retransmisión de mensajes entre clientes de una misma sala.
- **Cliente:** Proporciona la interfaz gráfica y coordina la interacción del usuario con la red a través de pantallas y gestores especializados.
- **Gestores:** Encapsulan la lógica de captura de hardware (cámara, micrófono) y transferencia de archivos, separando la complejidad técnica de la interfaz de usuario.

12.3. Clases Principales

A continuación se presentan las clases más importantes del sistema, su responsabilidad y sus métodos principales:

Clase	Responsabilidad	Métodos principales
Servidor	Abre socket TCP, acepta conexiones y administra clientes	iniciar(), detener(), reenviar_a_sala()
ManejadorCliente	Atiende un cliente en un hilo separado, interpreta y procesa mensajes	iniciar(), _procesar_mensaje(), _enviar()
ClienteSocket	Abstrae el socket TCP del cliente con callbacks asíncronos	conectar(), enviar(), enviar_y_recibir(), registrar_callback()
BaseDatos	Singleton que maneja la conexión SQLite y las consultas	conectar(), inicializar(), hash_clave()
AuthService	Valida credenciales de usuario contra la base de datos	validar_login()
MainApp	Ventana raíz de CustomTkinter, administra transiciones entre pantallas	_mostrar_login(), _mostrar_principal(), _mostrar_sala()
GestorVideo	Captura y transmite video desde la cámara web	toggle_camara(), _capturar_y_enviar(), on_camera_frame()
GestorAudio	Captura audio del micrófono y reproduce audio recibido	toggle_mic(), _capturar_y_enviar_audio(), on_audio_frame()
GestorArchivos	Transfiere archivos en fragmentos codificados en Base64	_enviar_archivo(), on_file_start(), on_file_end()

Cuadro 12: Clases principales del sistema

12.4. Flujo de Ejecución

El siguiente diagrama describe la secuencia general del programa desde su inicio hasta la interacción en una sala:

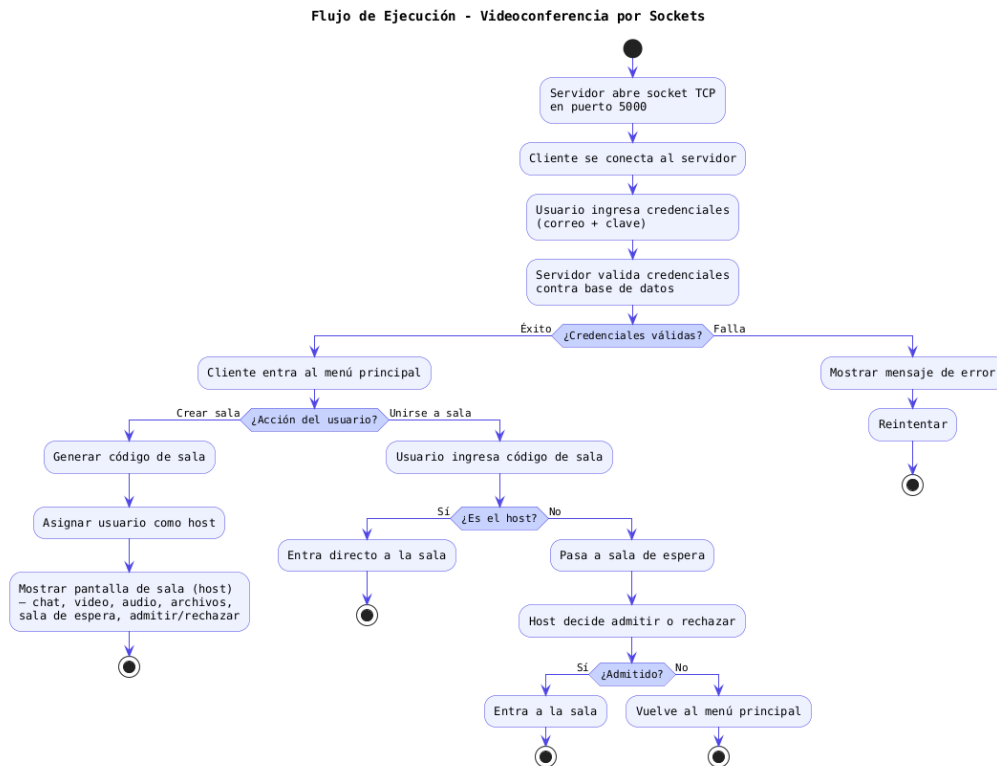


Figura 7: Flujo de ejecución general del sistema

12.5. Funcionamiento de las Clases y Componentes

El sistema está dividido en tres componentes principales: el Lado del Servidor, el Lado del Cliente y los Gestores de Hardware/Archivos.

12.5.1. Lado del Servidor

El servidor gestiona la persistencia de datos y la comunicación concurrente entre clientes:

- **Protocolo** (`Servidor/protocolo.py`): Define las constantes de mensaje que viajan serializadas en formato JSON sobre el socket TCP, sirviendo como diccionario del protocolo.
- **BaseDatos** (`Servidor/base_datos.py`): Inicializa y mantiene la base de datos SQLite. Centraliza las consultas, registros de salas y autenticaciones.
- **AuthService** (`Servidor/auth_service.py`): Valida las credenciales de los usuarios comparando el hash SHA-256 almacenado con el ingresado por el cliente.

- **ManejadorCliente** (`Servidor/manejador_cliente.py`): Representa el hilo asignado por el servidor para atender a un cliente. Recibe e interpreta los mensajes JSON delimitados por `\n` y despacha las peticiones.
- **Servidor** (`Servidor/servidor.py`): Clase principal que abre el socket TCP de escucha, acepta nuevas conexiones de clientes y crea un hilo daemon con su respectivo **ManejadorCliente**.

El servidor utiliza un bucle infinito para aceptar conexiones entrantes, creando un hilo independiente para cada cliente. Esto permite atender múltiples usuarios simultáneamente:

```
1 # Servidor/servidor.py - Bucle de aceptacion de clientes
2 while self._activo:
3     conexion, direccion = self._socket.accept()
4     manejador = ManejadorCliente(self, conexion, direccion)
5     with self._lock:
6         self._clientes.append(manejador)
7     hilo = threading.Thread(target=manejador.iniciar, daemon=True)
8     hilo.start()
```

12.5.2. Lado del Cliente

El cliente proporciona la interfaz gráfica en CustomTkinter y procesa la interacción con el usuario y la red:

- **MainApp** (`Cliente/main.py`): Ventana raíz de CustomTkinter que administra la transición fluida entre las diferentes vistas de la aplicación.
- **ClienteSocket** (`Cliente/cliente_socket.py`): Abstrae el socket del cliente, manteniendo un hilo de escucha para procesar mensajes entrantes en tiempo real.
- **Modelos** (`Cliente/modelos/usuario.py`, `mensaje.py`): Objetos que modelan a un usuario conectado y los mensajes del chat, facilitando su transporte y manipulación.
- **Pantallas** (`LoginFrame`, `PantallaPrincipal`, `PantallaSala`, `PanelVideo`): Subclases de CustomTkinter especializadas en dibujar los formularios de login, el menú de salas, la sala de reuniones y las pantallas de video respectivamente.

12.5.3. Gestores de Red y Hardware

Clases auxiliares delegadas que encapsulan la complejidad de manejar hardware y transmisión:

- **GestorVideo** (`Cliente/gestores/gestor_video.py`): Controla la captura de la cámara web, decidiendo si usa OpenCV o FFmpeg, y envía los frames comprimidos y codificados en Base64.
- **GestorAudio** (`Cliente/gestores/gestor_audio.py`): Captura audio en PCM usando `sounddevice` y reproduce el audio recibido en un hilo y cola FIFO independientes.
- **GestorArchivos** (`Cliente/gestores/gestor_archivos.py`): Controla la transferencia segmentada de archivos en Base64, partiendo el archivo en trozos de 1500 bytes y reconstruyéndolo en el receptor.

12.6. Aplicación de los Pilares de la POO

Los pilares de la Programación Orientada a Objetos se aplican directamente en la estructuración de la aplicación para resolver problemas de modularidad y acoplamiento:

12.6.1. Encapsulamiento

Se utiliza para proteger el estado interno de los objetos, ocultando los sockets directos, colecciones de clientes o contraseñas, y exponiéndolas solo mediante métodos o propiedades controladas. Por ejemplo, en la clase `Usuario` (`Cliente/modelos/usuario.py`), los datos del perfil están protegidos con prefijos de guion bajo, y el acceso se restringe mediante decoradores `@property`. En el servidor, la lista de clientes conectados se protege detrás de un lock para evitar condiciones de carrera:

```
1 class Usuario:
2     def __init__(self, id_usuario, nombres, correo, rol):
3         self._id_usuario = id_usuario # Atributo privado
4         self._nombres = nombres
5         self._rol = rol
6
7     @property
8     def id_usuario(self):
9         return self._id_usuario
```

```
10
11     def es_host(self):
12         return self._rol.lower() == "host"
```

12.6.2. Herencia

Se implementa principalmente en la interfaz gráfica para reutilizar los componentes visuales de CustomTkinter. Clases como `MainApp` heredan de `ctk.CTk`, y las pantallas (`LoginFrame`, `PantallaPrincipal`, `PantallaSala`, `PanelVideo`) heredan de `ctk.CTkFrame`, permitiendo extender y personalizar el comportamiento de contenedores gráficos estándar:

```
1 class PanelVideo(ctk.CTkFrame): # Hereda de ctk.CTkFrame
2     def __init__(self, master, usuario_nombre="", width=240, height=180,
3         bg="#111111"):
4         super().__init__(master, width=width, height=height, fg_color=bg
5             , corner_radius=8, border_width=1, border_color="#2d2d2d")
6         self.pack_propagate(False)
7         # Widgets especificos para dibujar el video
```

12.6.3. Polimorfismo

Permite tratar de forma homogénea diferentes pantallas dentro del flujo de la aplicación. En `MainApp`, la variable `_frame_actual` puede contener instancias de `LoginFrame`, `PantallaPrincipal` o `PantallaSala`. Al cambiar de pantalla, el contenedor llama polimórficamente a los métodos heredados como `pack()` y `destroy()`:

```
1 # Transicion dinamica de pantallas en MainApp
2 self._frame_actual.destroy() # Destruye la pantalla previa
3 self._frame_actual = PantallaSala(self, ...) # Instancia nueva pantalla
4 self._frame_actual.pack(fill=tk.BOTH, expand=True) # Empaqueta
   polimorficamente
```

12.6.4. Abstracción

Oculta la complejidad técnica de la red y el hardware de las pantallas de usuario. `ClienteSocket` encapsula toda la lógica del socket TCP, serialización JSON y buffer, ofreciendo solo métodos de alto nivel como `enviar` o `registrar_callback`:

```
1 class ClienteSocket:
2     def conectar(self): ...
3     def enviar(self, datos):
4         self._socket.send((json.dumps(datos) + "\n").encode("utf-8"))
5     def registrar_callback(self, tipo, callback):
6         self._callbacks[tipo] = callback
```

12.7. Patrones de Diseño Implementados

Los patrones de diseño se utilizaron para dar una solución limpia y estandarizada a las problemáticas típicas de una aplicación cliente-servidor:

12.7.1. Singleton (Instancia Única)

Garantiza que la clase `BaseDatos` posea una única instancia en todo el ciclo del servidor, centralizando las conexiones y evitando fallos por concurrencia en SQLite:

```
1 class BaseDatos:
2     _instancia = None
3
4     def __new__(cls, *args, **kwargs):
5         if cls._instancia is None:
6             cls._instancia = super().__new__(cls)
7         return cls._instancia
```

12.7.2. Observer (Observador)

El `ClienteSocket` actúa como sujeto que recibe mensajes de la red, y las diferentes pantallas o gestores se registran como observadores a través de callbacks, recibiendo notificaciones cuando se detecta un mensaje que les compete:

```
1 # Suscripcion (Observadores se registran)
2 self._cliente_socket.registrar_callback("CHAT_MESSAGE", self._recibir_chat)
3
4 # Notificacion en ClienteSocket
5 def _escuchar(self):
6     ...
```

```
7     cb = self._callbacks.get(mensaje.get("type"))
8     if cb:
9         cb(mensaje) # Notifica al callback registrado
```

12.7.3. Strategy (Estrategia)

El `GestorVideo` define algoritmos de captura de video intercambiables. Si el cliente dispone de OpenCV instalado en su sistema, el gestor utiliza `cv2.VideoCapture` para capturar los frames; en caso contrario, alterna a una estrategia basada en `ffmpeg` leyendo directamente desde el dispositivo de video local:

```
1 def _capturar_y_enviar(self):
2     if self._backend == "opencv":
3         self._capturar_con_opencv()
4     else:
5         self._capturar_con_ffmpeg()
```

12.7.4. Facade (Fachada)

El `ManejadorCliente` en el servidor funciona como una fachada que centraliza y simplifica el uso de múltiples subsistemas complejos del servidor (como `AuthService`, `BaseDatos` y la distribución de mensajes de `Servidor`), exponiendo solo una interfaz simple mediante el método `iniciar()`.

12.7.5. Command (Comando)

El protocolo de red encapsula las intenciones del usuario en objetos JSON que viajan por el socket con un campo `type` que actúa como comando. Esto permite enrutar y procesar de forma genérica las peticiones:

```
1 # Servidor interpreta y despacha comandos de forma desacoplada
2 if tipo == Protocolo.LOGIN_REQUEST:
3     self._manejar_login(mensaje)
4 elif tipo == Protocolo.CREATE_ROOM:
5     self._manejar_crear_sala(mensaje)
```

12.7.6. Factory Method (Método Fábrica)

`MainApp` delega la creación e instanciación de las pantallas visuales a métodos fábrica específicos (`_mostrar_login()`, `_mostrar_principal()`, `_mostrar_sala()`). El cliente no conoce cómo se construyen las pantallas, solo invoca la fábrica:

```
1 def _mostrar_sala(self, codigo_sala, es_host, id_sala):
2     self._frame_actual = PantallaSala(self, self._usuario, self.
3         _cliente_socket, codigo_sala, es_host, id_sala, self._salir_sala)
4     self._frame_actual.pack(fill=tk.BOTH, expand=True)
```

12.7.7. Template Method (Método Plantilla)

En la transmisión de video, la captura e inicialización siguen un esqueleto predefinido (establecer conexión, inicializar buffer, iterar captura, codificar, enviar y liberar recursos). La lógica del bucle de captura se define en la superclase del gestor, mientras que los pasos específicos para la lectura de la cámara cambian según la estrategia.

12.7.8. Resumen de Patrones

Patrón	Clase principal	Uso
Singleton	<code>BaseDatos</code>	Garantiza una única instancia de conexión a la base de datos SQLite.
Observer	<code>ClienteSocket</code>	Callbacks por tipo de mensaje; las pantallas se suscriben y reciben notificaciones.
Strategy	<code>GestorVideo</code>	Selecciona dinámicamente entre OpenCV y FFmpeg según el backend disponible.
Facade	<code>ManejadorCliente</code>	Simplifica el acceso a <code>AuthService</code> , <code>BaseDatos</code> y el relay de mensajes.
Command	Protocolo JSON	El campo <code>type</code> del mensaje actúa como comando para enrutar en el servidor.
Factory Method	<code>MainApp</code>	Métodos <code>_mostrar_*</code> () crean las diferentes pantallas sin acoplar al flujo.
Template Method	<code>GestorVideo</code>	Define el esqueleto de captura y envío; las subclases implementan la lectura.

Cuadro 13: Resumen de patrones de diseño implementados

12.8. Manejo de Errores

El sistema controla diversas situaciones de error para garantizar una experiencia de usuario estable:

- **Pérdida de conexión del cliente:** El hilo listener detecta cuando `recv()` devuelve datos vacíos y marca la conexión como cerrada. El servidor elimina al cliente de la lista de conexiones activas.
- **Credenciales incorrectas:** `AuthService` retorna un mensaje con `status: error` que se muestra en la interfaz de login sin revelar información sensible.
- **Código de sala inválido:** Al unirse a una sala, si el código no existe en la base de datos, el servidor responde con un mensaje de error que el cliente muestra al usuario.
- **Archivos rechazados:** Si el receptor rechaza la descarga, el archivo no se guarda en ese cliente y se muestra un mensaje informativo.
- **Errores de cámara o micrófono:** Los gestores capturan excepciones al abrir los dispositivos de hardware y notifican al usuario mediante mensajes en la interfaz. Si no se detecta micrófono, se muestra un mensaje indicándolo.
- **Timeout en operaciones:** `enviar_y_recibir()` tiene un timeout de 10 segundos. Si el servidor no responde en ese tiempo, se lanza una excepción `ConnectionError` que la interfaz captura y muestra.

```
1 # Ejemplo: manejo de error en login
2 try:
3     respuesta = self._cliente_socket.enviar_y_recibir({
4         "type": "LOGIN_REQUEST",
5         "correo": correo,
6         "clave": clave
7     })
8     if respuesta.get("status") == "success":
9         # Transicion a pantalla principal
10    else:
11        self._mensaje_error.config(text=respuesta.get("message", "Error_
12            desconocido"))
13 except ConnectionError as e:
14     self._mensaje_error.config(text=str(e))
```

12.9. Pruebas Realizadas

Se realizaron pruebas funcionales para verificar el correcto funcionamiento de los componentes principales del sistema:

Prueba	Procedimiento	Resultado esperado
Login correcto	Ingresar credenciales válidas (luis@email.com / 123456)	Acceso al sistema, muestra pantalla principal
Login incorrecto	Ingresar credenciales inválidas	Mensaje de error, permanece en login
Crear sala	Presionar “Crear Sala” desde el menú principal	Sala registrada en BD, ingresa como host
Unirse a sala válida	Ingresar código de sala existente	Solicitud enviada al host, espera admisión
Unirse a sala inválida	Ingresar código inexistente	Mensaje “Sala no encontrada”
Enviar mensaje	Escribir mensaje y presionar Enter	Todos los participantes reciben el mensaje
Compartir archivo	Seleccionar archivo desde el diálogo	Archivo fragmentado, enviado y reconstruido en receptor
Activar cámara	Presionar “Iniciar Cámara”	Frame capturado, codificado, enviado y mostrado en los demás clientes
Activar micrófono	Presionar “Iniciar Micrófono”	Audio capturado y transmitido en tiempo real
Admitir usuario (host)	Seleccionar solicitante y presionar “Admitir”	Usuario admitido, ingresa a la sala
Rechazar usuario (host)	Seleccionar solicitante y presionar “Rechazar”	Usuario notificado, regresa al menú
Salir de sala	Presionar “Salir de la Sala”	Cámara/mic detenidos, callbacks limpiados, vuelve al menú

Cuadro 14: Pruebas funcionales del sistema

13. Concurrencia

El sistema emplea el módulo `threading` de Python para manejar múltiples operaciones simultáneamente sin bloquear la interfaz de usuario ni el servidor.

13.1. Modelo de Concurrencia del Servidor

Cada cliente conectado genera un hilo independiente (`ManejadorCliente`) que corre en un bucle infinito escuchando mensajes. Esto permite que varios usuarios puedan usar el sistema simultáneamente sin bloquear la ejecución principal:

```
Servidor (hilo principal, bucle accept)
|
|-- Thread: Cliente 1 (ManejadorCliente)
|     Lee mensajes, procesa, responde
|
|-- Thread: Cliente 2 (ManejadorCliente)
|     Lee mensajes, procesa, responde
|
|-- Thread: Cliente 3 (ManejadorCliente)
|     Lee mensajes, procesa, responde
```

El servidor protege las estructuras compartidas (lista de clientes, estado de salas) con un `threading.Lock` para evitar condiciones de carrera al acceder desde múltiples hilos.

13.2. Concurrency en el Cliente

El cliente utiliza hilos `daemon` para las siguientes tareas:

- **Hilo listener:** Escucha mensajes entrantes del servidor en segundo plano.
- **Hilo de captura de video:** Captura frames de la cámara en un bucle cada 100 ms.
- **Hilo de captura de audio:** Lee bloques del micrófono y los envía al servidor.
- **Hilo de reproducción de audio:** Lee de una cola FIFO y escribe en el altavoz.
- **Hilo de envío de archivos:** Lee y fragmenta el archivo en segundo plano.

Todos los hilos del cliente son `daemon=True`, lo que asegura que se cierren automáticamente al cerrar la ventana principal, evitando hilos huérfanos.

14. Seguridad

El sistema implementa medidas de seguridad básicas para proteger la información de los usuarios:

14.1. Almacenamiento de Contraseñas

Las contraseñas nunca se almacenan en texto plano. Se utiliza el algoritmo **SHA-256** (FIPS 180-4) para generar un hash hexadecimal de 64 caracteres que se guarda en la base de datos:

```
1 @staticmethod
2 def hash_clave(clave):
3     return hashlib.sha256(clave.encode()).hexdigest()
```

Cuando un usuario inicia sesión, el sistema calcula el hash de la contraseña ingresada y lo compara con el hash almacenado. Si coinciden, el acceso es concedido. Esto garantiza que incluso si la base de datos se ve comprometida, las contraseñas originales no quedan expuestas.

Sin embargo, se debe advertir que el prototipo presenta los siguientes riesgos de seguridad:

- **Riesgo de texto plano:** Si se almacenasen contraseñas en texto plano (un error crítico que debe evitarse), cualquier persona con acceso físico o lógico a la base de datos o al disco podría comprometer directamente la totalidad de las cuentas.
- **Falta de Sal (Salt):** Al no utilizar un valor de sal aleatorio único para cada usuario antes de calcular el hash, el sistema es vulnerable a ataques de diccionario y tablas de arcoiris (rainbow tables) si la base de datos es expuesta.

14.2. Validación de Autenticidad

Todas las operaciones sensibles (crear sala, unirse a sala, admitir usuarios) requieren que el usuario haya iniciado sesión previamente. El servidor mantiene el estado de autenticación de cada conexión y rechaza peticiones de usuarios no autenticados.

Sin embargo, es importante señalar que las comunicaciones no están cifradas (no se usa TLS/SSL), no hay protección contra ataques de fuerza bruta en el login, y los tokens de sesión son implícitos (la conexión TCP misma define la sesión). Estas son limitaciones a considerar para una versión de producción.

15. Limitaciones Actuales

El sistema presenta las siguientes limitaciones identificadas, que representan oportunidades de mejora para versiones futuras:

- **Latencia en video y audio:** Debido a que el sistema utiliza TCP para transportar video y audio, puede presentar mayor latencia comparado con protocolos especializados como UDP/RTP. TCP reintenta la transmisión de paquetes perdidos, lo que puede causar retrasos acumulativos en tiempo real.
- **Ancho de banda:** Los frames de video se transmiten como JPEG en Base64, lo que aumenta el tamaño del mensaje aproximadamente un 33% respecto al binario original. Para 10 fps con calidad 50, cada frame puede ocupar entre 10 KB y 50 KB.
- **Escalabilidad:** El servidor usa un hilo por cliente, lo que limita el número de conexiones simultáneas según la capacidad del hardware. Con 100+ clientes concurrentes, el cambio de contexto puede degradar el rendimiento.
- **Almacenamiento de archivos:** Los archivos se cargan completamente en memoria antes de enviarse, lo que limita el tamaño máximo a la memoria RAM disponible. No hay streaming ni carga progresiva.
- **Seguridad:** No hay cifrado de extremo a extremo ni protección contra ataques de intermediario (MITM) en la capa de transporte.
- **Historial de chat:** Los mensajes de chat se almacenan en la base de datos pero no se recuperan al ingresar a una sala (no hay persistencia de historial visible para el usuario).
- **Concurrencia de Base de Datos:** Al usar SQLite, el sistema cuenta con bloqueos de escritura a nivel de base de datos, lo cual limita la escalabilidad en un entorno concurrente masivo.
- **Rendimiento de Interfaz (CustomTkinter):** El motor de renderizado de la UI de CustomTkinter no está diseñado para procesar y redibujar flujos de video de alta definición o alto frame rate de manera eficiente en la CPU.

16. Protocolos y Normativas Utilizados

El sistema utiliza **TCP** como protocolo de transporte para todas las comunicaciones. Cada funcionalidad prepara el dato de forma distinta antes de enviarlo, pero todas comparten el mismo canal confiable.

- **Video:** la cámara es capturada por OpenCV (o ffmpeg). El frame se comprime en formato JPEG y se convierte a texto mediante Base64 para poder viajar dentro de un mensaje JSON. Elegimos TCP para transportar esos frames entre máquinas — aunque en sistemas reales como Zoom se usa UDP para menor latencia.
- **Audio:** el micrófono entrega bloques de audio crudo (PCM). Esos bytes también se convierten a Base64 y se envían por TCP. El receptor los reproduce en orden mediante una cola interna.
- **Archivos:** el archivo se lee completo, se codifica en Base64 y se corta en pedazos pequeños. Se envían con un mensaje de inicio, varios fragmentos y uno de fin — así el receptor sabe cuándo rearmar el archivo original.
- **Mensajes de control** (login, crear sala, chat, etc.): viajan directamente como objetos JSON sobre TCP, delimitados por el carácter `\n`.

16.1. Tabla de Mensajes del Protocolo

Los mensajes se envían en JSON sobre TCP, delimitados por `\n`.

17. Manual de Instalación y Ejecución

17.1. Requisitos

El prototipo requiere la instalación de dependencias de Python y de librerías del sistema operativo para el correcto funcionamiento de la interfaz gráfica, la transmisión de audio y la captura de video:

17.1.1. 1. Librerías de Python

Instale las dependencias de Python ejecutando:

Tipo	Dirección	Campos
LOGIN_REQUEST	Cliente → Servidor	correo, clave
LOGIN_RESPONSE	Servidor → Cliente	status, idUsuario, nombres, rol
CREATE_ROOM	Cliente → Servidor	codigo, nombre, idHost
JOIN_ROOM_REQUEST	Cliente → Servidor	codigo, idUsuario
WAITING_ROOM_UPDATE	Servidor → Host	idSala, solicitanteId, solicitanteNombre
ADMIT_USER	Host → Servidor	idSala, idUsuario
REJECT_USER	Host → Servidor	idSala, idUsuario
CHAT_MESSAGE	Cliente → Sala	roomCode, userName, message
FILE_START	Cliente → Sala	roomCode, fileName, fileSize
FILE_CHUNK	Cliente → Sala	roomCode, fileName, data
FILE_END	Cliente → Sala	roomCode, fileName
CAMERA_FRAME	Cliente → Sala	roomCode, userName, data (base64)
VIDEO_START	Cliente → Sala	roomCode, userName
VIDEO_STOP	Cliente → Sala	roomCode, userName
LEAVE_ROOM	Cliente → Servidor	roomCode

Cuadro 15: Tipos de mensaje del protocolo

```
1 pip install -r requirements.txt
```

Esto instalará:

- **Pillow:** Para el procesamiento y renderizado de frames de imagen en CustomTkinter.
- **sounddevice:** Para la captura y reproducción de audio en tiempo real.
- **opencv-python:** Para la captura y compresión de frames de video desde la cámara.
- **customtkinter:** Para la interfaz gráfica moderna de usuario.

17.1.2. 2. Dependencias del Sistema

Adicionalmente, se requieren dependencias a nivel de sistema operativo para el soporte de la interfaz, el audio y el video:

- **En Linux (Debian/Ubuntu):** Ejecute el siguiente comando para instalar el soporte base de Tkinter (requerido internamente por CustomTkinter), PortAudio (requerido por *sounddevice*) y FFmpeg (como alternativa de video):

```
1 sudo apt update
2 sudo apt install python3-tk libportaudio2 ffmpeg -y
```

- **En macOS:** Instale las dependencias a través de Homebrew:

```
1 brew install portaudio ffmpeg
```

- **En Windows:** Las librerías de Python usualmente empaquetan las dependencias binarias (como PortAudio) automáticamente en sus ruedas (*wheels*), por lo que no requiere configuración adicional si utiliza la cámara con OpenCV.

17.2. Ejecución

1. Iniciar el servidor:

```
1 python -m Servidor.servidor
```

2. Iniciar uno o más clientes:

```
1 python -m Cliente.main
```

17.3. Datos de Prueba

Nombre	Correo	Clave	Rol
Luis Tasayco	luis@email.com	123456	Host
Ana Torres	ana@email.com	123456	Usuario
Carlos Ruiz	carlos@email.com	123456	Usuario

Cuadro 16: Usuarios de prueba

18. Conclusiones

- Se implementó un prototipo funcional de videoconferencia con arquitectura cliente-servidor.
- Los sockets TCP con protocolo JSON proporcionan una base sólida para comunicación en tiempo real.
- Los cuatro pilares de la POO – encapsulamiento, herencia, polimorfismo y abstracción – fueron aplicados consistentemente en todo el código.
- Se identificaron y aplicaron siete patrones de diseño: Singleton, Observer, Strategy, Facade, Command, Template Method y Factory Method.
- La POO permitió organizar el código en clases especializadas con responsabilidades claras.
- La concurrencia con hilos permite atender múltiples clientes simultáneamente.
- La transmisión de video se logró mediante captura, compresión JPEG y codificación base64.

19. Anexos

19.1. Repositorio del Proyecto

El código fuente completo del prototipo, incluyendo los scripts de base de datos, la documentación y el código del cliente y del servidor, se encuentra disponible públicamente en el siguiente repositorio de GitHub:

- https://github.com/LaTP16/Zoom_Simulador